

Trim — Automatic Bank Import + Trim Premium Billing

Design spec · 2026-07-15 · Status: approved design, not yet built Read with: ARCHITECTURE.md, SECURITY.md, FEATURES.md. This spec follows all three; where it adds rules, it says so.

1. What we're building and why

Two independent subsystems, built in this order:

- **Subsystem A — Bank sync.** Purchases made on a user's existing debit/credit cards appear in Trim automatically, via open banking. Kills the #1 budgeting-app complaint: having to type in every expense.
- **Subsystem B — Trim Premium billing.** Stripe-powered subscription (~£3.99/mo) so Trim can be shared beyond friends and earn money. Manual logging stays free forever; bank sync becomes the premium feature *later* — during the current testing phase it is free for everyone.

The key correction from the validation phase: Stripe cannot see purchases made on users' existing cards — it processes payments *to* businesses, it has no visibility into anyone's spending elsewhere. Automatic import requires an **open banking data provider**. Stripe's role in Trim is billing only.

2. Validation summary (researched 2026-07-15)

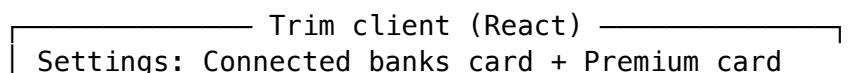
Question	Verdict	Evidence
Do people want auto-import?	Yes	It's the feature every paid budgeting app charges for: YNAB \$109/yr, Monarch \$99.99/yr, Copilot \$95/yr, Snoop £5.99/mo, Emma £0–14.99/mo. Mint's 2024 shutdown left a gap.
Will it work technically?	UK: yes. US: yes (later adapter). Vietnam: no.	No aggregator covers VN consumer banks. VND users keep manual logging with friendly messaging.

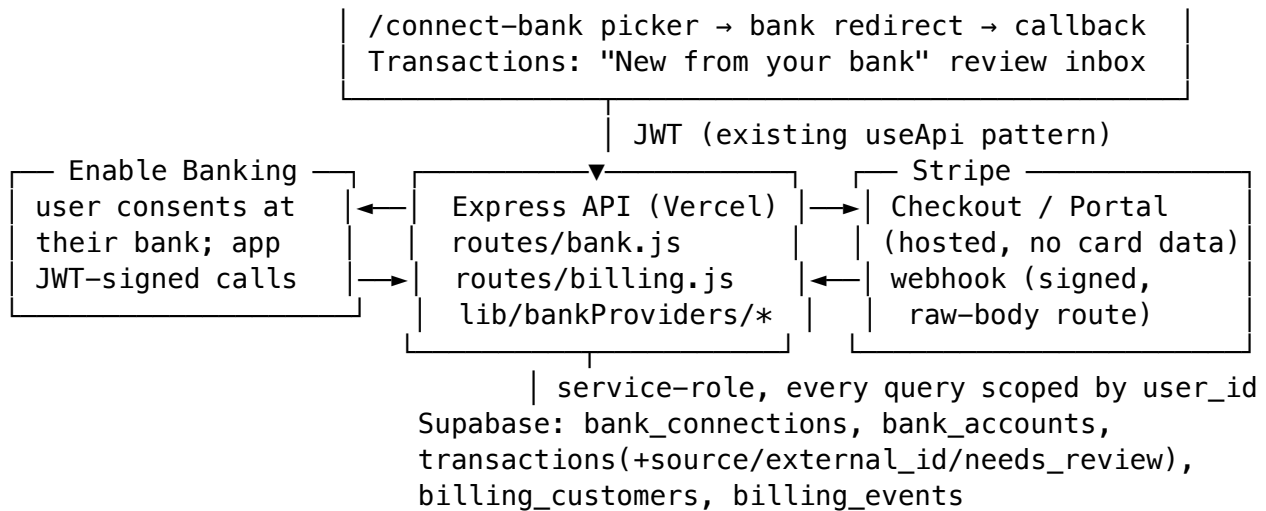
Question	Verdict	Evidence
Which provider?	Enable Banking (UK/EU)	The classic indie option — GoCardless Bank Account Data (ex-Nordigen) — closed to new signups July 2025 . Plaid in Europe is enterprise-sales only. Stripe Financial Connections is US bank accounts only . Enable Banking has self-serve signup, free sandbox, and free “restricted production” (whitelist your own real accounts) — perfect for the friends-testing phase. Paid agreement needed for public scale (terms sales-gated — confirm in the A0 spike). Users authenticate <i>at their bank</i> (redirect); Trim receives read-only data through the provider and never sees bank credentials. Card payments use Stripe-hosted Checkout; card numbers never touch Trim’s servers. §7.
Is it secure?	Yes, by construction	
Unit economics?	Freemium required at scale	Connected banks cost money per user per month at scale; Stripe takes ~1.5% + 20p (UK consumer cards) plus Billing fees. Gating sync behind ~£3.99/mo keeps cost aligned with revenue; free (manual) users cost ≈ £0.

Decisions locked in (Alex, 2026-07-15): target banks UK + US (+ VN unsupported); freemium *later*, sync free during testing; bank sync built before billing.

Honest risks: bank-connection reliability is every budgeting app’s #1 support burden; UK consent re-authorisation (~90/180 days) is recurring friction; Enable Banking’s whitelisting limits for friends’ accounts and production pricing are unverified until A0; auto-import weakens the manual-logging streak loop — addressed by the review inbox (§5.6).

3. Architecture overview





Everything follows the existing contract: the browser never talks to Supabase for data; all new routes live behind `requireAuth`; every query is scoped by `req.user.id`; RLS on every new table as defence-in-depth.

Naming rule: the billing subsystem is called **billing / plan / premium** in all code and copy. The word “subscriptions” is taken — `server/lib/subscriptions.js` + the Subscriptions page mean *detected recurring merchant charges*, a completely different feature.

4. How the Enable Banking flow works (primer)

Design-level understanding; exact endpoint names/fields must be verified against current Enable Banking docs during A0.

1. **App registration (one-time, manual):** Alex creates an Enable Banking account, registers an application, and receives an application ID + downloads a **private signing key**. All API calls are authenticated by a JWT that our server signs with this key — there is no per-user OAuth client secret dance.
2. **Connect:** server asks Enable Banking to start an authorization for a chosen bank (ASPSP), passing our redirect URL and a state value. User is redirected to their **bank's own** login/consent screen (possibly via their banking app).
3. **Callback:** bank redirects the user's browser back to our client callback page with a code + our state. Client posts these (authenticated with the user's Trim JWT) to our API, which exchanges the code for a **session** — the handle for the consented accounts. Consent has an expiry (UK typically ~90 or 180 days), after which the user must re-authorise.
4. **Fetch:** with the session, the server lists accounts (IBAN/identifier, currency, name) and fetches transactions per account with date-range filters. We import **booked** (posted) transactions only — pending ones mutate and are skipped in MVP.
5. **Restricted production:** the free mode. Accounts must be whitelisted in the Enable Banking control panel; API returns data only for whitelisted accounts. Fine for Alex + a handful of friends (limits to be confirmed in A0); full production needs a commercial agreement.

5. Subsystem A — bank sync

5.1 Schema — server/migrations/010_bank_sync.sql

Follows the house migration style (plain SQL, RLS + service-role grants like 001–009). Sketch:

```
create table bank_connections (
  id                uuid primary key default gen_random_uuid(),
  user_id           uuid not null references auth.users(id) on delete cascade,
  provider          text not null default 'enablebanking',
  provider_session_id text,                -- set after completeAuth
  institution_id    text not null,         -- provider's ASPSP id
  institution_name   text not null,
  status            text not null default 'pending'
  check (status in ('pending','active','expired','revoked','error')
  auth_state        text,                  -- HMAC-signed state for the in-flight
  consent_expires_at timestampz,
  last_synced_at    timestampz,
  last_sync_error    text,
  created_at        timestampz not null default now()
);

create table bank_accounts (
  id                uuid primary key default gen_random_uuid(),
  connection_id     uuid not null references bank_connections(id) on delete cascade,
  user_id           uuid not null references auth.users(id) on delete cascade,
  provider_account_uid text not null,
  display_name       text not null,        -- e.g. "Monzo Current Account"
  currency           currency_code not null,
  enabled            boolean not null default true,
  created_at        timestampz not null default now(),
  unique (connection_id, provider_account_uid)
);

alter table transactions
  add column source      text not null default 'manual' check (source in ('manual',
  add column bank_account_id uuid references bank_accounts(id) on delete set null,
  add column external_id  text,
  add column needs_review boolean not null default false;

-- dedup guard: one imported row per provider transaction
create unique index transactions_import_dedup
  on transactions (user_id, bank_account_id, external_id)
  where external_id is not null;

-- + RLS (auth.uid() = user_id) and service_role grants on both new tables,
--   and an index on transactions (user_id, needs_review) where needs_review
```

Notes: - external_id = the provider's stable transaction id; when a bank doesn't supply one, the

adapter synthesises sha256(accountUid|bookingDate|amount|description) — imperfect but deterministic. - Deleting a connection **keeps** imported transactions (they're the user's history); bank_account_id nulls out via on delete set null cascade path (accounts cascade with the connection, transactions survive).

5.2 Provider adapter — server/lib/bankProviders/

```
bankProviders/  
├─ index.js           # getProvider(name) registry + shared normalizers  
└─ enablebanking.js  # first concrete adapter
```

Every adapter implements the same five functions and returns **normalized shapes only** (no provider payloads leak upward):

```
listInstitutions(country)           // → [{ id, name, logoUrl }]
startAuth({ institutionId, redirectUrl, state }) // → { authUrl }
completeAuth({ code })              // → { sessionId, consentExpiresAt, accounts: [norm
fetchAccounts(sessionId)           // → [{ uid, displayName, currency }]
fetchTransactions(sessionId, accountUid, { since }) // → [normalizedTx], booked only

// normalizedTx = { externalId, amount (positive number), type: 'income'|'expense',
//                  description, date: 'YYYY-MM-DD', currency }
```

A future stripeFC.js (US, Stripe Financial Connections) or plaid.js implements the same interface — routes and pipeline don't change. Enable Banking JWT signing (its private key) lives in this module only.

5.3 Sync engine — server/lib/bankSync.js

No reliable cron on Vercel Hobby, so the trigger is **sync-on-app-open**:

- Client fires POST /api/bank/sync once after login/dashboard mount (fire-and-forget; react-query invalidates on completion).
- Server **throttle**: skip any connection synced within the last 6 hours unless { force: true } (the UI's "Sync now" button). Force is rate-limited (§5.7).
- Per connection: fetch transactions per enabled account from last_synced_at – 3 days (overlap window; dedup absorbs the repeats). First sync after connect backfills **90 days**.
- Volumes are personal-banking sized (tens of rows) — comfortably inside Vercel's 60 s ceiling. Connections are processed sequentially; each account's import is one batched insert.
- Failure handling: per-connection try/catch; store last_sync_error, set status='error' (or 'expired' when the provider says consent lapsed), continue with other connections. UI shows a soft "reconnect" prompt — never an alarming red state.

5.4 Import pipeline (inside bankSync)

For each fetched normalized transaction: 1. **Currency guard**. Trim is single-currency-per-user with no FX. At connect time, accounts whose currency ≠ user_stats.currency are marked enabled = false and the UI explains why ("This account is in EUR — Trim tracks your budget in GBP"). The pipeline also skips any stray mismatched row. 2. **Dedup**. Insert with on conflict do nothing against transactions_import_dedup. 3. **Auto-categorize**.

Reuse the existing suggestion logic (same as GET /api/categories/suggest): user's own history first, then the lib/categoryKeywords.js keyword map. High/medium confidence ☐ set category_id; low ☐ leave null. (Refactor note: extract the route's lookup into server/lib/categorySuggest.js so route and pipeline share one implementation.) 4. **Insert** with source='import', needs_review=true, **without** touching gamification — applyLogEvent must NOT run on imports (a 90-day backfill would otherwise mint months of fake streaks/XP).

5.5 Routes — server/routes/bank.js (all requireAuth + requirePremium, Zod-validated)

Route	Body / params	Behaviour
GET /api/bank/institutions?country= GB country=GB)	country in supported set (GB)	Proxied provider list, cached in-memory ~24 h.
POST /api/bank/connections	{ institutionId }	Creates pending connection; generates single-use state = HMAC(connectionId, userId, nonce) stored on the row; returns { connectionId, authUrl }.
POST /api/bank/connections/:id/complete	{ code, state }	Verifies state matches the pending row (single-use, then cleared), exchanges code ☐ session, stores accounts (currency-guarded), sets active, runs first sync inline. Generic 400 on any mismatch.
GET /api/bank/connections	—	Connections + accounts + status + consent_expires_at for the Settings card.
DELETE /api/bank/connections/:id	UUID guard	Best-effort provider session revoke; delete row (accounts cascade; transactions kept).
POST /api/bank/sync	{ force? }	Runs the sync engine for the user; returns { imported, skipped, connections: [...status] }.

requirePremium (new, server/middleware/plan.js): reads the user's plan (\$6) — but while PREMIUM_ENFORCED=false (today), it always passes. Flipping to freemium later is a config change, not a rebuild.

5.6 Review inbox — the engagement answer

Auto-import would kill Trim's core loop (streaks/XP are earned by logging). The replacement daily habit is **reviewing**:

- Imported transactions surface at the top of the Transactions page in a “**New from your bank (N)**” section, newest first, suggested category chip pre-selected.
- **One tap ☐ confirms** (needs_review=false). Tapping a different category chip recategorises *and* confirms in the same tap. 3-tap rule intact.
- **Gamification:** the first review action of the day fires the existing applyLogEvent once (streak + one +10 XP award) — the daily engagement act survives, nothing inflates. Implemented in the review endpoint, keyed off user_stats.last_logged_date.
- Emptying the inbox triggers a small celebration (confetti burst + “All caught up ☐ ”). Dashboard shows a soft badge (“3 new from your bank”) linking to the inbox. Friendly, never nagging.
- Review = PATCH /api/transactions/:id extended to accept { needsReview: false, categoryId? }, plus POST /api/transactions/review-all for one-tap bulk confirm of high-confidence rows.

5.7 Rate limits & client pieces

- New limiters in server/index.js: bankSyncLimiter 10/hour/user (keyed on req.user.id), bankConnectLimiter 10/15 min/user.
- Client: Settings gains a **Connected banks** card (list, status pills, Sync now, Connect a bank, Disconnect). New route /connect-bank (institution picker) + /connect-bank/callback (parses ?code&state, posts to complete, celebrates, deep-links to the inbox). Query keys: ['bank', 'connections']; sync invalidates ['transactions'], ['dashboard'], ['subscriptions'], ['analytics'].
- **Unsupported countries:** if user_stats.currency is VND (or the user has no supported-country bank), the card shows “Bank sync isn’t available for banks in Vietnam yet — Quick-Add has your back ☐ ” and hides the connect CTA. AUD likewise until an AU provider exists. Copy stays playful, never apologetic-corporate.

6. Subsystem B — Trim Premium billing (Stripe)

6.1 Schema — server/migrations/011_billing.sql

```
create table billing_customers (
  user_id          uuid primary key references auth.users(id) on delete cascade,
  stripe_customer_id text unique not null,
  plan            text not null default 'free' check (plan in ('free', 'premium'
  stripe_subscription_id text,
  subscription_status text,           -- raw Stripe status mirror (active, past_due
  current_period_end  timestampz,
  updated_at         timestampz not null default now()
);

create table billing_events (           -- webhook idempotency ledger
  event_id  text primary key,         -- Stripe evt_... id
  type      text not null,
  received_at timestampz not null default now()
);
-- + RLS on billing_customers (auth.uid() = user_id); billing_events is server-internal
```

6.2 Routes — server/routes/billing.js (requireAuth)

- GET /api/billing □ { plan, status, currentPeriodEnd, enforced } for the Settings card.
- POST /api/billing/checkout □ find-or-create the Stripe Customer (store mapping), create a **Checkout Session** (mode: 'subscription', price STRIPE_PRICE_ID, client_reference_id = user.id, success/cancel URLs on CLIENT_URL/settings?billing=...), return { url }. Client just window.location = url — hosted page, **no Stripe.js, no CSP changes, no card data near Trim**.
- POST /api/billing/portal □ Customer Portal session { url } (cancel, change card, invoices — all Stripe-hosted).

6.3 Webhook — POST /api/stripe/webhook

- Mounted in server/index.js **before** the global express.json({limit: '100kb'}) (currently ~line 55) with express.raw({ type: 'application/json' }), outside requireAuth — the first deliberate unauthenticated write route; authentication is Stripe's signature: stripe.webhooks.constructEvent(req.body, sig, STRIPE_WEBHOOK_SECRET), generic 400 on failure.
- **Idempotency**: insert event_id into billing_events first; conflict □ 200 and skip (Stripe retries deliveries).
- Events handled: checkout.session.completed (link customer □ user via client_reference_id, set plan premium), customer.subscription.updated / customer.subscription.deleted / invoice.payment_failed (mirror status; plan follows active/trialing □ premium, else free).

6.4 Client + pricing

- Settings gains a **Trim Premium** card: plan badge, “Upgrade” □ checkout redirect, “Manage billing” □ portal redirect. When PREMIUM_ENFORCED=true and plan is free, the Connected-banks card renders its friendly paywall state instead.
- Launch price: **£3.99/mo or £29/yr** (undercuts Snoop £5.99; covers per-user provider cost with margin — final call after A0 reveals Enable Banking's real production pricing).
- Rollout: Stripe **test mode** end-to-end first (test cards via the stripe:test-cards skill), live keys only in session B2; friends grandfathered by manually setting plan='premium'.

7. Security addendum (fold into SECURITY.md when building)

Surface	Threat	Mitigation
Bank consent flow	credential phishing	Users authenticate at their bank ; Trim never sees or handles bank credentials. Data is read-only AIS.

Surface	Threat	Mitigation
Callback	CSRF / connection hijack	Single-use HMAC state bound to user+connection, verified and cleared server-side; generic 400 on mismatch.
Connection/account ids	IDOR	UUID guard + every query <code>.eq('user_id', req.user.id)</code> + RLS — same belt-and-braces as all existing tables.
Provider secrets	key theft	Enable Banking private key + session ids server-only (env + DB); never logged, never sent to the browser; browser sees institution names/labels only.
Stripe webhook	forgery / replay	Signature verification on the raw body; <code>billing_events</code> idempotency ledger; no detail in error responses.
Card data	PCI exposure	None touches Trim — hosted Checkout + Portal only.
Sync endpoint	cost/API-quota DoS	Per-user throttle (6 h) + <code>bankSyncLimiter</code> 10/h + <code>bankConnectLimiter</code> 10/15 min.
Import writes	data poisoning across users	Pipeline writes are scoped to the connection's <code>user_id</code> ; dedup index is per-user.

New env vars — **server only** (server/.env, Vercel project settings): `ENABLEBANKING_APP_ID`, `ENABLEBANKING_PRIVATE_KEY` (or key path), `BANK_STATE_HMAC_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_ID`, `PREMIUM_ENFORCED` (default false). Nothing new in client env. Fail-fast checks at boot for whichever subsystem is enabled.

8. Build phases (one session each; Trim's definition of done applies to every row)

#	Session	Ships / proves	Manual steps for Alex
A0	Spike — go/no-go. Enable Banking account; sandbox flow; restricted production with Alex's own bank; scratch script (scratchpad, not committed) pulls real transactions. Confirm: friends-whitelisting limits, production pricing, UK regulatory-umbrella terms, exact API shapes for \$4.	Real UK bank data flows before any product code. Kill/pivot cheaply if blocked.	Create Enable Banking account; register app; download private key; whitelist own bank account.
A1	Migration 010 + adapter + connect flow: Settings card, /connect-bank picker, callback, complete.	Connect a real bank from the running UI.	Run 010 in Supabase SQL editor; add env vars.
A2	Sync engine + import pipeline: throttle, 90-day backfill, dedup, currency guard, auto-categorize, categorySuggest extraction.	Real purchases appear in Trim untouched by hand; re-sync imports no duplicates.	—
A3	Review inbox + gamification wiring + inbox-zero celebration + dashboard nudge + unsupported-country UX.	The daily loop feels good; VND users get graceful messaging.	—

#	Session	Ships / proves	Manual steps for Alex
B1	Stripe test mode end-to-end: migration 011, checkout, portal, webhook (raw-body mount fix in <code>index.js</code>), Premium card, <code>requirePremium</code> middleware (unenforced).	Test card up-grades/downgrades a real account; webhook idempotency proven.	Create Stripe account; create Product+Price (test); add webhook endpoint; set env vars.
B2	Go live: live keys + price, Stripe account activation, flip <code>PRE-MIUM_ENFORCED=true</code> when ready to charge, grandfather friends.	First real revenue.	Activate Stripe account (business details); live webhook; decide final price.
C (later)	US adapter (<code>stripeFC.js</code> via Stripe Financial Connections, or <code>plaid.js</code> pay-as-you-go) behind the same interface.	US friends get sync.	Provider account + keys.

Per-session definition of done (house rules): `npm run build` passes in `client/`; the feature is reachable by clicking through the running UI; `BUILD_PLAN.md` + `FEATURES.md` (+ `SECURITY.md` when auth/data-access changed) updated in the same session; work committed; if in a worktree, explicitly flagged as needing merge to main.

9. Open items to resolve in A0

1. Enable Banking restricted-production limits: how many accounts can be whitelisted, and can friends' accounts be added?
2. Production pricing + the commercial/regulatory path to public UK launch (their AIS umbrella terms).
3. Exact consent lifetime per major UK bank (drives the re-auth reminder UX).
4. Whether Enable Banking returns stable transaction ids for the target banks (drives how often the hash fallback runs).
5. Vercel Hobby cron limits — whether a daily top-up sync is worth adding in A2.

10. Sources (validation research, 2026-07-15)

- Stripe Financial Connections docs — US-account scope: <https://docs.stripe.com/financial-connections>

- GoCardless Bank Account Data new-signups-disabled notice: <https://bankaccountdata.gocardless.com/new-signups-disabled>
- Plaid pricing (Europe = custom plans): <https://plaid.com/pricing/> · <https://support.plaid.com/hc/en-us/articles/16110502116887>
- Enable Banking restricted production / FAQ: <https://enablebanking.com/docs/faq/>
- Indie open-banking landscape 2026: <https://www.openbankingcompare.com/blog/best-open-banking-api-providers-for-developers-2026> · <https://www.openbankingtracker.com/guides/free-open-banking-apis>
- Competitor pricing: <https://walletgrower.com/compare/ynab-vs-monarch-vs-copilot> · <https://www.nimblefins.co.uk/Budgeting-apps-review-emma-YNAB-snoop> · <https://moneytothemasses.co.uk/savings/tips/the-best-budgeting-apps-in-the-uk-how-to-budget-without-trying>
- TrueLayer pricing (sales-gated production): <https://truelayer.com/data/> · <https://blog.finexer.com/truelayer-pricing-uk/>